# 引き継ぎ書: 業務工数システムを受け取る人が、最初に読む巻

> この巻は、教育資料（00〜14）とは別に、**このシステムを「仕事として」引き継ぐ**ために要ることを、1 冊にまとめた物です。プログラミングの知識はまだ要りません。分からない言葉が出てきたら、「10_用語集」を引いてください。
>
> 書いてあることは、アプリのプログラム（GitHub の `master` ブランチ）を読んで確かめた事実です。プログラムから分からないこと（人・お金・決まりごと）は、推測で書かずに「§10 前任者に確かめること」に集めました。

### この巻で分かること

- このシステムの全体（本番・開発・使う人）を 1 枚の図で
- 引き継ぎで **受け取る物** のチェックリストと、前任者が **渡す前にやること**
- 最初の 1 日・1 週間・1 か月に、何をすればよいか
- 変更が **本番に出る仕組み**（`master` に push すると、そのまま本番へ出ます）
- **画面とプログラムの対応表**（全 49 画面。「この画面を直したい」時に、どのファイルを開くか）
- **押す前に必ずバックアップを取る操作** と、手元でだけ書き換えてコミットしない所
- よくある依頼（人が入った・辞めた・ログインできない など）への対応のしかた

---

## 目次


1. まず、全体を 1 枚で
2. だれが、何をするシステムか
3. 引き継ぎで受け取る物
4. 前任者が、引き継ぐ前にやること
5. 最初の 1 か月の進め方
6. 変更を本番に出す流れ
7. 画面とプログラムの対応表
8. 気を付けること
9. よくある依頼と、対応のしかた
10. 前任者に確かめること
11. この巻のまとめ


---

# 1. まず、全体を 1 枚で

このシステム（業務工数システム）は、現場の従業員が「いつ・どの作業に・何時間使ったか（＝工数）」を記録し、班やショップ（職場）ごとに集計するための Web アプリです。現場の人はブラウザで本番のアプリを毎日使い、あなた（保守の担当）は、自分の PC でプログラムを直して、本番に出します。

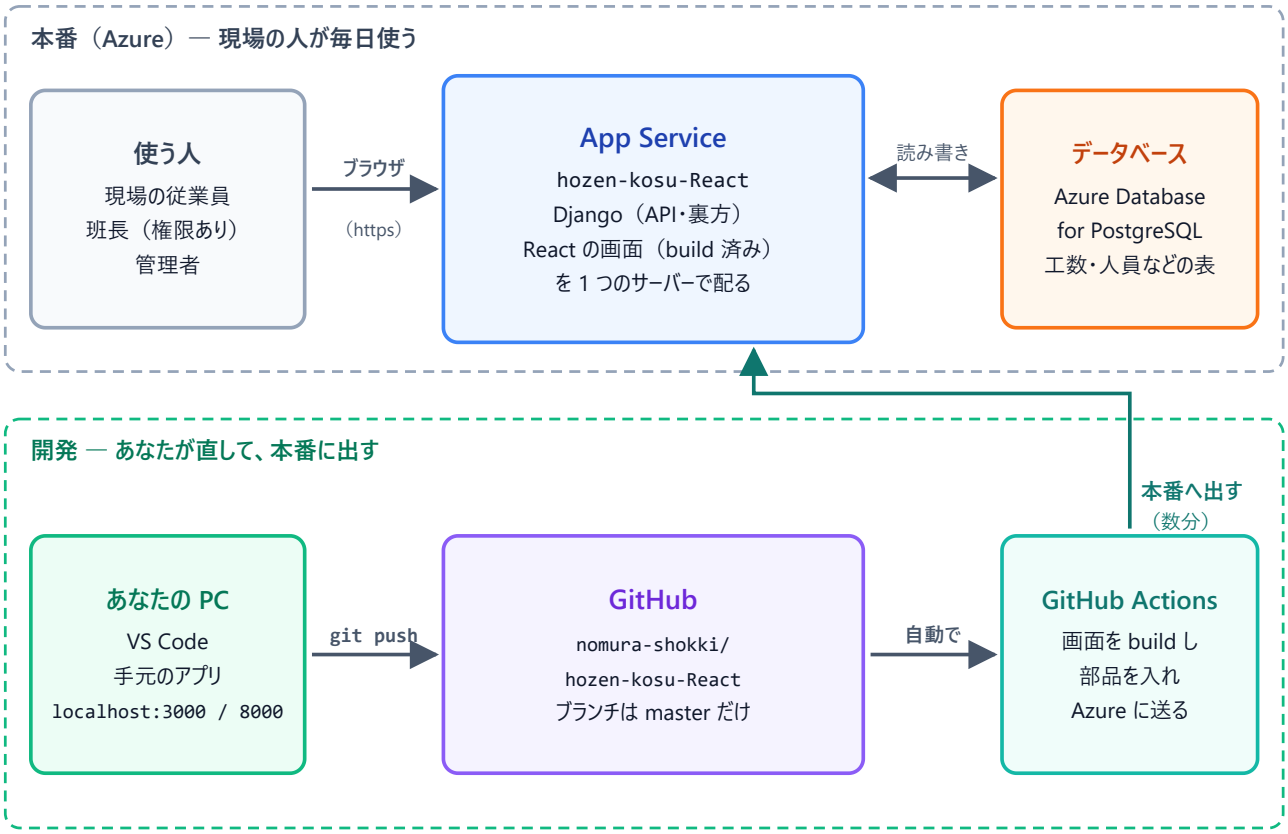



図1: 引き継ぐ物の全体図。上が本番（使う人が毎日使う所）、下が開発（あなたが直す所）。master に push すると、GitHub Actions が自動で本番に出す

| 物 | どこ・何 | くわしくは |
|---|---|---|
| **本番のアプリ** | `https://hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net/`（Microsoft Azure の App Service `hozen-kosu-React`。`japaneast` は東日本の地域） | 09 章 §13 |
| **本番のデータベース** | Azure Database for PostgreSQL（フレキシブルサーバー）。住所（`DB_HOST`）やパスワードは秘密の値なので、リポジトリには書いてありません | 09 章 §13.3 |
| **プログラムの倉庫** | GitHub の `nomura-shokki/hozen-kosu-React`。ブランチは `master` の 1 本だけ | 02 章 |
| **本番に出す仕組み** | GitHub Actions（`.github/workflows/master_hozen-kosu-react.yml`）。`master` に push すると、自動で本番へ出ます | 本巻 §6・09 章 §14 |
| **手元の置き場所** | 前任者の PC では `kosu_react` フォルダの中の `hozen-kosu-React/`。あなたは 01 章 §7 の `git clone` で、自分の PC に作ります | 01 章 |
| **使っている技術** | 画面: React 19 ＋ TypeScript（`frontend/`）／サーバー: Django 4.2 ＋ Django REST Framework（`kosu/`・`hozen_another/`）／データベース: PostgreSQL | 00 章・11 章 |
| **この資料** | `kosu/` フォルダ（PDF 版・Web 版つき）。Web 版は `https://oltotlo79-rgb.github.io/education/kosu/html/` | README |

> **本番・開発とは？**「本番」は、現場の人が実際に使っている所です。ここを壊すと、現場の仕事が止まります。「開発」は、あなたの PC の中で、壊しても困らない所です。直す時は、必ず開発（手元）で試してから本番に出します（09 章 §2）。

---

## 2. だれが、何をするシステムか

### 2.1 使う人と、使えるメニュー

使う人は、人員の表（`member`）に登録されている従業員です。人員の表の「**権限**」と「**管理者権限**」の 2 つのスイッチで、メインメニューに出るボタンが増えます。

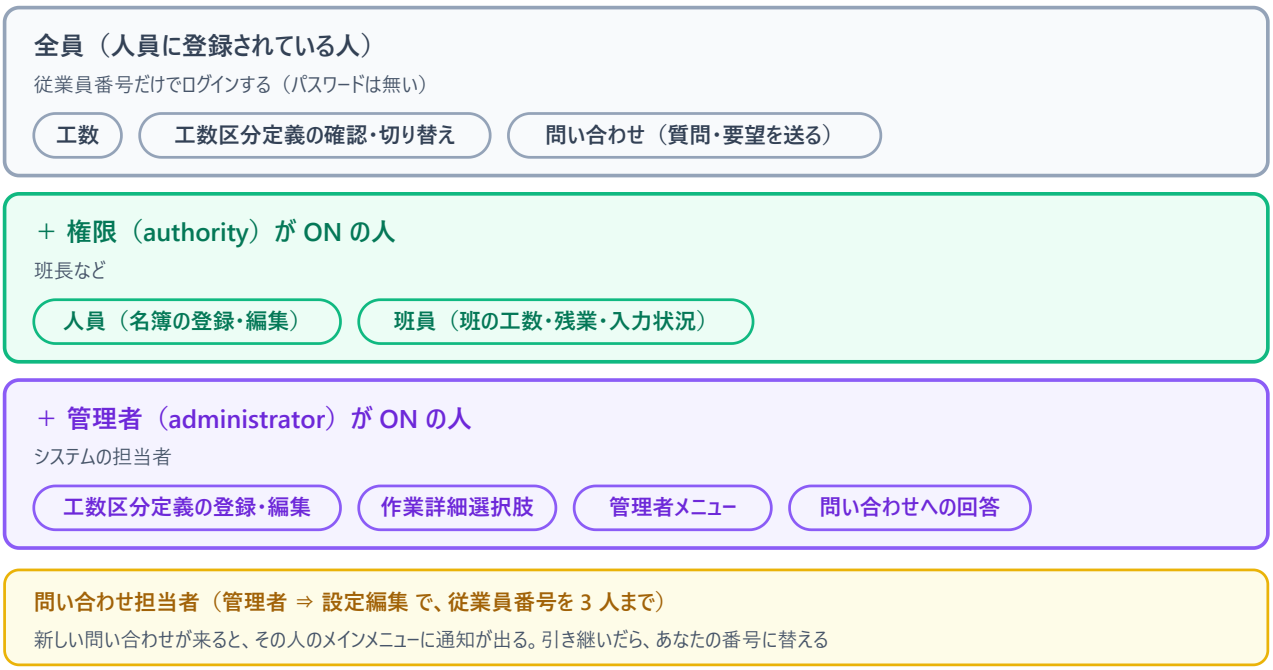



図2: 使う人と、使えるメニュー。人員の表の「権限」と「管理者」の 2 つのスイッチで、メインメニューに出るボタンが増える（2 つは別々のスイッチ）

| 人 | 人員の表のスイッチ | メインメニューに出るボタン |
|---|---|---|
| 全員 | （なし） | 工数・工数区分定義（確認と切り替えだけ）・問い合わせ |
| 班長など | **権限**（`authority`）が ON | 上に加えて、人員・班員 |
| システムの担当者 | **管理者権限**（`administrator`）が ON | 上に加えて、管理者。工数区分定義の登録・編集、作業詳細選択肢、問い合わせへの回答もできる |
| 問い合わせ担当者 | 管理者 ⇒ 設定編集 の「問い合わせ担当者従業員番号 1〜3」 | 新しい問い合わせが来ると、メインメニューに通知が出る |

- **2 つのスイッチは別々です。**「管理者権限」だけ ON で「権限」が OFF の人は、管理者メニューは出ますが、人員・班員のボタンは出ません。あなた自身は、両方 ON にしてもらいます（§3）。
- スイッチは、権限のある人が **人員** ⇒ **人員一覧** ⇒ **編集** の「権限:」「管理者権限:」で付け外しします。

## 2.2 ログインのしかた（パスワードは無い）

- ログインの画面で **従業員番号だけ** を入れます。人員の表にその番号があれば、だれでもその人としてログインできます（パスワードはありません）。
- 一度ログインすると、約 10 年ログインしたままになります（`settings.py` の `SESSION_COOKIE_AGE = 315360000` 秒）。共用の PC では、使い終わったらメインメニューの「ログアウト」を押してもらいます。
- 番号が人員の表に無いと「入力された従業員番号は登録がありません。ERROR048」、工数区分定義が 1 つも無いと「利用可能な工数区分がありません。ERROR052」と出て、ログインできません。

> **だから「権限」と「管理者権限」を付ける人は、必要な人だけにします。**従業員番号を知っていれば、その人としてログインできてしまうからです。

### 2.3 通知（ポップアップ）のしくみ

メインメニューの上に出る青い字のお知らせは、次の時に出ます。押すと問い合わせの内容が開き、通知は消えます。

| いつ | だれに | 出る文 |
|---|---|---|
| 新しい問い合わせが来た | 問い合わせ担当者（設定編集で決めた 3 人） | 「〇〇さんからの新しい問い合わせがあります。」 |
| 問い合わせが編集された | 問い合わせ担当者 | 「ID〇の問い合わせが編集されました。」 |
| 問い合わせに回答が付いた | 問い合わせた本人 | 「ID〇の問い合わせに回答が来ています。」 |

> ⚠️ **通知は、1 人 5 件までです**（人員の表と設定の表の `pop_up1` 〜 `pop_up5`）。5 件たまっていると、新しい通知は出ません。問い合わせ担当者は、通知をためずに開いて消してください。

---

## 3. 引き継ぎで受け取る物

前任者から、次の物を受け取ります。☐ にチェックを入れながら進めてください。**値（パスワードなど）は、この資料にもチャットにも書きません。**

| ☐ | 受け取る物 | 何に使う | 気を付けること |
|---|---|---|---|
| ☐ | **GitHub のアカウント**と、`nomura-shokki/hozen-kosu-React` に push できる権限 | プログラムを直して本番に出す | Secrets（下の行）を見る・変えるには、リポジトリの管理者（Admin）の権限が要る |
| ☐ | **GitHub の Secrets 8 つの値**（または作り直す方法） | 本番に出す時に GitHub Actions が使う | 名前は `SECRET_KEY`・`DEBUG`・`DB_NAME`・`DB_USER`・`DB_PASSWORD`・`DB_HOST`・`HELP_PATH`・`AZUREAPPSERVICE_PUBLISHPROFILE_…`（Azure に送るための発行プロファイル）。置き場所は、リポジトリの **Settings → Secrets and variables → Actions**。保存した値は二度と見られないので、前任者がどこに控えているかを聞く |
| ☐ | **Azure ポータルのアカウント**と権限 | 本番のサーバーとデータベースを見る・設定する | 対象は App Service `hozen-kosu-React` と PostgreSQL のサーバー（あれば、ステージングの `hozen-kosu-another` も）。お金（サブスクリプション）の担当がだれかも聞く |
| ☐ | **App Service の「環境変数」の中身**（名前と、値の控えの場所） | 本番で動いているアプリが読む秘密の値 | リポジトリに `.env` は無く、Secrets は本番に出す間だけ使う物なので、本番のアプリは Azure の「環境変数」（以前の画面では「構成」→「アプリケーション設定」）から読んでいるはず。Azure ポータルで確かめる |
| ☐ | **本番データベースにつなぐ許可** | 本番のデータを直接調べる・migrate する時 | Azure のデータベースのファイアウォールに、あなたの場所の IP アドレスを入れてもらう必要があるかもしれない（09 章 §13.3） |
| ☐ | **本番アプリでの、あなたの人員データ** | 管理者のメニューに入る | あなたの従業員番号を人員に登録し、「権限」と「管理者権限」を ON にしてもらう |
| ☐ | **問い合わせ担当者を、あなたの番号に** | 新しい問い合わせの通知を受け取る | 管理者 ⇒ 設定編集 ⇒「問い合わせ担当者従業員番号 1〜3」 |
| ☐ | **ヘルプ画面のパスワード**（`HELP_PATH`） | ヘルプ画面（`/help`）でデータを初期化する時 | 初期化は、ほとんど使わない危ない操作（§8.2） |
| ☐ | **Django の管理画面（`/admin/`）のアカウント**（あれば） | データベースの中身を画面で直接見る | 01 章 §13 の `createsuperuser` で作る物。本番に作ってあるかは、リポジトリからは分からない |
| ☐ | **前任者の手元にしかない物** | — | まだ記録（コミット）していない変更、バックアップの Excel、手順のメモ、未回答の問い合わせ・やりかけの改修の一覧 |

> 🔒 **秘密の値の受け取り方。** パスワード・秘密の鍵は、メールやチャットに貼らずに、会社で決まった方法（パスワード管理の道具など）で受け取ります。受け取ったあと、落ち着いたら、前任者が知ったままでなくてよい物（ `HELP_PATH` ・データベースのパスワード・発行プロファイル）は変えることを考えます。変える時は、**使っている所を全部** 書き換えます（例: データベースのパスワードなら、Azure のデータベース・App Service の環境変数・GitHub の Secrets の 3 か所）。1 か所でも古いままだと、本番が動かなくなったり、本番に出せなくなったりします。

---

## 4. 前任者が、引き継ぐ前にやること

この節は、**渡す側（前任者）** 向けのチェックリストです。後任の方は、前任者といっしょに確かめてください。

| ☐ | やること | なぜ |
|---|---|---|
| ☐ | 手元のアプリのフォルダで `git status` を打ち、**記録していない変更が無いか** 確かめる。要る物はコミットし、要らない物は捨てる | 前任者の PC にしかない変更は、引き継げない |
| ☐ | 特に、**手元でだけ書き換えた 2 か所**（§8.1 の `frontend/.env.production` と `hozen_another/settings.py` ）が、本番の形に戻っているか確かめる | 戻さずに push すると、本番の画面が使えなくなる |
| ☐ | 手元にしかないファイル（バックアップの Excel など）のうち、要る物を渡す | 個人の情報（氏名・従業員番号）が入っているので、会社で決まった方法で |
| ☐ | GitHub・Azure の権限を、後任に付ける（§3） | 後任が本番に出せるように |
| ☐ | 本番アプリで、後任の人員データを作り、「権限」「管理者権限」を ON にし、問い合わせ担当者を後任の番号にする | 後任が管理者のメニューに入れるように |
| ☐ | 「§10 前任者に確かめること」の答えを書いて渡す | プログラムには書いていないことだから |
| ☐ | 引き継ぎが終わった後、前任者の権限（GitHub・Azure・アプリの管理者権限）をどうするか決める | 使わない権限を残さないため |

---

## 5. 最初の 1 か月の進め方

はじめは「見るだけ」、次に「手元で動かす」、最後に「前任者といっしょに本番へ出す」の順に進めます。

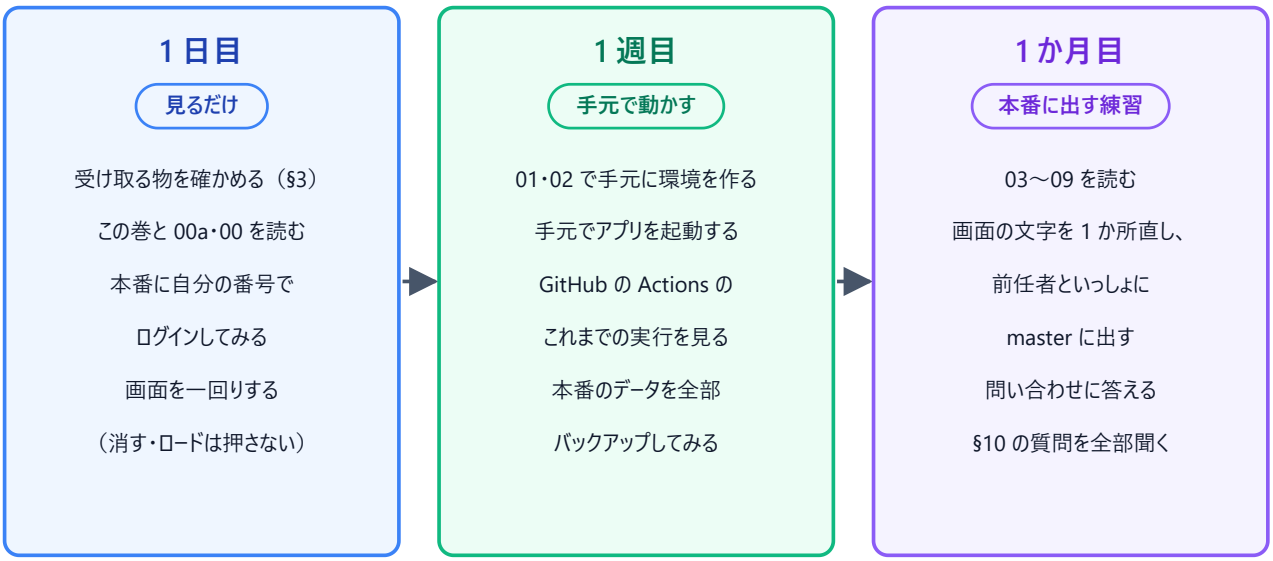



図3: 引き継いでから 1 か月の進め方。はじめは「見るだけ」、次に「手元で動かす」、最後に「前任者といっしょに本番へ出す」

## 5.1 1 日目: 見るだけ

- ☐ §3 の「受け取る物」を、前任者といっしょに確かめる。
- ☐ この巻と、「00a_はじめての方へ_超入門」「00_導入と全体像」を読む（分からない所は飛ばしてよい）。
- ☐ 本番のアプリに、自分の従業員番号でログインしてみる。
- ☐ §7 の表を見ながら、すべてのメニューを一回りする。「**削除**」「**ロード**」「**初期化**」**は押さない**（§8.2）。

## 5.2 1 週目: 手元で動かす

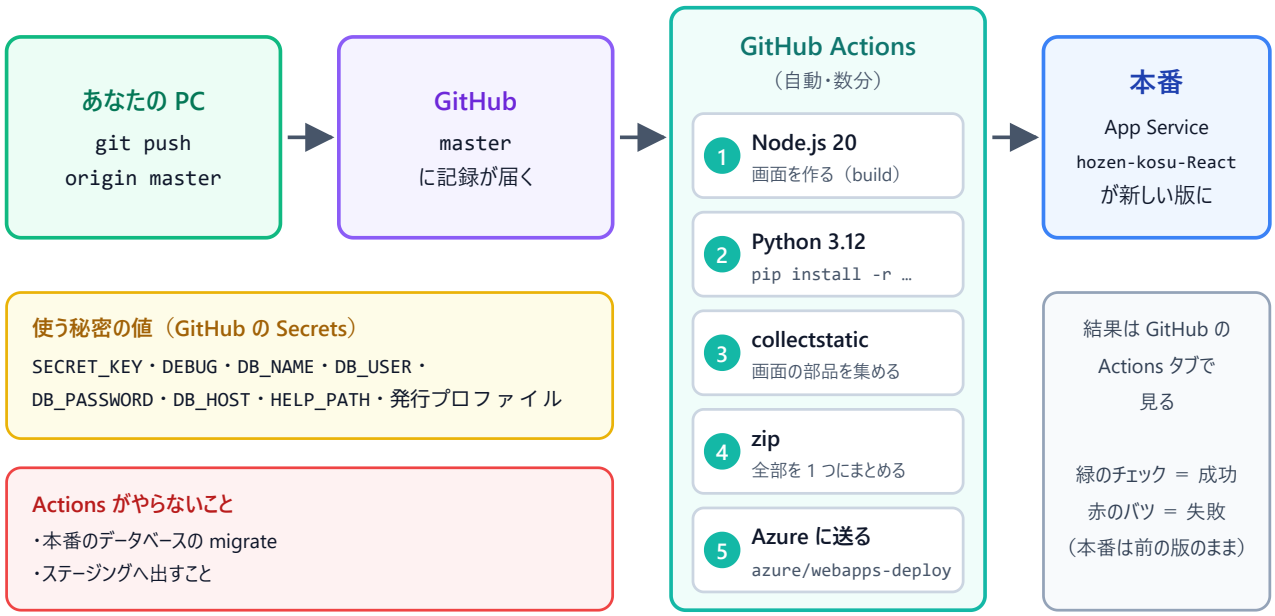
- ☐ 「01_環境構築」と「02_Git_GitHub」を見ながら、自分の PC に開発の環境を作り、アプリを `git clone` する。
- ☐ 手元でアプリを起動し、ログインの画面を出す（01 章のゴール）。
- ☐ GitHub のリポジトリの **Actions** タブを開き、これまでの「Deploy Django + React to Azure」の実行の記録（緑のチェック・赤のバツ）を見る。
- ☐ 本番の 管理者 ⇒ データ管理 で、すべての種類のデータの「バックアップ開始」を 1 回ずつ押し、Excel を安全な場所に保管してみる（バックアップは読むだけの操作で、データは変わりません。工数は期間を選びます）。
- ☐ 「09_運用_保守_デプロイ」の §5〜§10（バックアップと復元）と §14（デプロイ）を読む。

## 5.3 1 か月目: 本番に出す練習

- ☐ 03〜08 章を読み、12 章でテストの流し方を覚える。
- ☐ 画面の文字を 1 か所だけ直し、手元で確かめて、前任者といっしょに `master` に出してみる（§6。業務の少ない時間に）。
- ☐ 問い合わせに、1 件答えてみる（§9.7）。
- ☐ 「§10 前任者に確かめること」を全部聞いて、答えを書き残す。
- ☐ 障害が起きた時の流れ（09 章 §16）を読み、だれに連絡するかを決めておく。

## 6. 変更を本番に出す流れ

これが、引き継ぎでいちばん大事な決まりです。




master に push した時のほか、Actions タブの「Run workflow」でも同じことが起きる

図5: 変更が本番に出る流れ。master に push すると、GitHub Actions が画面を作り直して Azure に送る。migrate はしない

- **`master` に push した瞬間に、本番へ出ていきます。**だれかがボタンを押す必要はありません。練習のつもりの push でも、本番が変わります。
- 画面の作り直し（`npm run build`）と部品集め（`collectstatic`）は、GitHub Actions が毎回やります。手元でやってコミットする必要はありません。
- **本番のデータベースの migrate は、自動ではありません。**モデル（表の形）を変えた時だけ、別に行います。その方法はリポジトリに書いていないので、前任者に確かめます（§10）。
- 結果は GitHub の **Actions** タブで見ます。緑のチェックなら成功、赤のバツなら失敗（本番は前の版のまま）です。
- 手で出し直したい時は、Actions タブで「Deploy Django + React to Azure」を選び、「Run workflow」を押します。

毎回の手順は、次の 7 つです（くわしくは 09 章 §14.2）。

| 順番 | やること | くわしくは |
|---|---|---|
| ① | 手元でテストを通す（`npm run test:run` と、バックのテスト） | 12 章 |
| ② | `git status` で、手元でだけ書き換えた 2 か所が戻っているか確かめる | 本巻 §8.1 |
| ③ | 変えたファイルを `git add` して、`git commit` する | 02 章 §6・§7 |
| ④ | `git push origin master`（＝本番に出る） | 02 章 §9 |
| ⑤ | Actions タブで、緑のチェックになるまで見守る（数分） | 09 章 §14.1 |
| ⑥ | モデルを変えた時だけ、本番のデータベースに migrate する | 09 章 §14.3 |
| ⑦ | 本番の画面で、ログイン・工数入力・直した所を確かめる | 09 章 §14.4 |

> ⚠️ **大きな変更は、業務の少ない時間に出します。** 出したら Actions が緑になるのを見届け、本番の画面で確かめるまでが、1 回のデプロイです。

---

# 7. 画面とプログラムの対応表

「この画面のここを直したい」と言われたら、まずこの表で、開くファイルを探します。

## 7.1 画面からデータまでの道筋

1 つの画面は、次の ①〜⑥ の順につながっています。

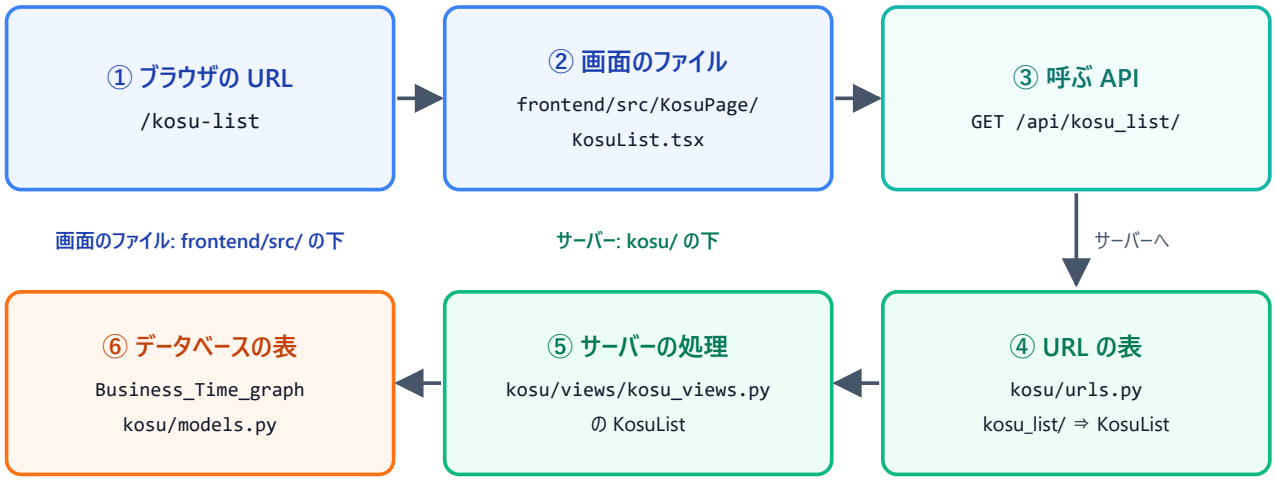



図4: 画面からデータまでの道筋（例: 工数一覧）。§7 の表は、この ①〜⑥ を画面ごとに並べた物。直したい所が「見た目」なら ②、「並び順・計算」なら ⑤、「表の列」なら ⑥

**表の見方:**

- **画面（タブの名前）と URL** … 上の行は、ブラウザのタブに出る名前です。下の行の URL は、ブラウザのアドレス欄の、住所の後ろの部分です。`:id` や `:employee_no` は、番号が入る所です（例: `/kosu-update/15`）。
- **画面のファイル** … 表の見出しの横に書いたフォルダの中にある、名前の後ろに `.tsx` が付いたファイルです（例: `KosuList` は `frontend/src/KosuPage/KosuList.tsx`）。画面の文字・ボタン・並びを直す時に開きます。
- **呼ぶ API** … その画面がサーバーに頼む時の住所です。実際の住所は `/api/kosu_list/` のように、前に `/api/`、後ろに `/` が付きます。
- **サーバーの処理** … `kosu/views/` の下のファイルと、その中のクラス（または関数）の名前です（`kosu_views.py`：`KosuList` は「`kosu/views/kosu_views.py` の中の `KosuList`」）。API の住所との対応は `kosu/urls.py` に書いてあります。一覧の並び順・しぼり込み・計算・権限の確かめを直す時に開きます。
- **使える人** … 「全員」はログインした人、「権限あり」は「権限」が ON の人、「管理者」は「管理者権限」が ON の人です（§2.1）。

## 7.2 全 49 画面の対応表

### ログイン・ヘルプ・メインメニュー（`frontend/src/MainPage/`）

| 画面（タブの名前）と URL | 画面のファイル（`.tsx`） | 呼ぶ API（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| ログイン<br>`/login` | `Login` | `login` | `main_views.py`：`Login` | だれでも |
| ヘルプ<br>`/help` | `Help` | `help_pass`・`help` | `main_views.py`：`HelpPass`・`Help` | だれでも（パスワード） |
| Main Menu<br>`/` | `MainMenu` | `main_menu`・`logout` | `main_views.py`：`Menu`・`Logout` | 全員 |

## 工数（自分の作業時間）（ `frontend/src/KosuPage/` ）

| 画面（タブの名前）とURL | 画面のファイル（ `.tsx` ） | 呼ぶ API（ `/api/…/` ） | サーバーの処理（ `kosu/views/` ） | 使える人 |
|---|---|---|---|---|
| 工数Menu `/kosu-menu` | `KosuMenu` | `kosu_menu` | `main_views.py`：`KosuMenu` | 全員 |
| 工数入力 `/kosu-new` | `KosuNew` | `kosu_new`・`set_day`・`over_time` | `kosu_views.py`：`KosuNew`・`SetDay`・`OverTime` | 全員 |
| 工数一覧 `/kosu-list` | `KosuList` | `kosu_list` | `kosu_views.py`：`KosuList` | 全員 |
| 工数編集 `/kosu-update/:id` | `KosuEdit` | `kosu_update`・`item_delete`・`day_update` | `kosu_views.py`：`KosuUpdate`・`ItemDlete`・`DayUpdate` | 全員 |
| 工数削除 `/kosu-delete/:id` | `KosuDelete` | `kosu_update`・`kosu_delete` | `kosu_views.py`：`KosuUpdate`・`KosuDelete` | 全員 |
| 当日休憩時間変更 `/today-break-time` | `TodayBreakTime` | `today_break_time` | `kosu_views.py`：`TodayBreakTime` | 全員 |
| 休憩時間変更 `/break-time` | `BreakTime` | `break_time` | `kosu_views.py`：`BreakTime` | 全員 |
| 勤務入力 `/kosu-calendar` | `KosuCalendar` | `kosu_calendar`・`kosu_calendar_change`・`kosu_work_write`・`work_default`・`tyoku_default`・`kosu_link` | `kosu_views.py`：`KosuCalendar`・`KosuCalendarChange`・`KosuWorkWrite`・`WorkDefault`・`TyokuDefault`・`KosuLink` | 全員 |

| 画面（タブの名前）と URL | 画面のファイル（`.tsx`） | 呼ぶ API（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| 工数集計 `/kosu-total` | `KosuTotal` | `kosu_total` | `kosu_views.py`：`KosuTotal` | 全員 |

### 工数区分定義（作業の種類の一覧）（`frontend/src/DefinitionPage/`）

| 画面（タブの名前）と URL | 画面のファイル（`.tsx`） | 呼ぶ API（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| 工数区分定義 Menu `/def-menu` | `DefMenu` | `def_menu` | `main_views.py`：`DefMenu` | 全員 |
| 工数区分定義切り替え `/def-search` | `DefSearch` | `def_search` | `def_views.py`：`DefSearch` | 全員 |
| 工数区分定義登録 `/def-new` | `DefNew` | `def_new` | `def_views.py`：`DefNew` | 管理者 |
| 工数区分定義一覧 `/def-list` | `DefList` | `def_list` | `def_views.py`：`DefList` | 管理者 |
| 工数区分定義編集 `/def-update/:id` | `DefEdit` | `def_update` | `def_views.py`：`DefUpdate` | 管理者 |
| 工数区分定義削除 `/def-delete/:id` | `DefDelete` | `def_update`・`def_delete` | `def_views.py`：`DefUpdate`・`DefDelete` | 管理者 |
| 工数区分定義切り替え `/def-ver` | `DefVer` | `def_ver` | `def_views.py`：`DefVer` | 全員 |
| 作業詳細選択肢登録 `/def-detail-new` | `DefDetailNew` | `def_detail_new` | `def_views.py`：`DefDetailNew` | 管理者 |
| 作業詳細選択肢一覧 `/def-detail-list` | `DefDetailList` | `def_detail_list` | `def_views.py`：`DefDetailList` | 管理者 |

| 画面（タブの名前）と URL | 画面のファイル（ `.tsx` ） | 呼ぶ API（ `/api/…/` ） | サーバーの処理（ `kosu/views/` ） | 使える人 |
|---|---|---|---|---|
| 作業詳細選択肢編集 `/def-detail-update/:id` | `DefDetailEdit` | `def_detail_update` | `def_views.py`：`DefDetailUpdate` | 管理者 |
| 作業詳細選択肢削除 `/def-detail-delete/:id` | `DefDetailDelete` | `def_detail_update`・`def_detail_delete` | `def_views.py`：`DefDetailUpdate`・`DefDetailDelete` | 管理者 |

## 人員（従業員の名簿）（ `frontend/src/MemberPage/` ）

| 画面（タブの名前）と URL | 画面のファイル（ `.tsx` ） | 呼ぶ API（ `/api/…/` ） | サーバーの処理（ `kosu/views/` ） | 使える人 |
|---|---|---|---|---|
| 人員Menu `/member-menu` | `MemberMenu` | `member_menu` | `main_views.py`：`MemberMenu` | 権限あり |
| 人員登録 `/member-new` | `MemberNew` | `member_new` | `member_views.py`：`MemberNew` | 権限あり |
| 人員一覧 `/member-list` | `MemberList` | `member_list` | `member_views.py`：`MemberList` | 権限あり |
| 人員編集 `/member-update/:employee_no` | `MemberEdit` | `member_update` | `member_views.py`：`MemberUpdate` | 権限あり |
| 人員削除 `/member-delete/:employee_no` | `MemberDelete` | `member_update`・`member_delete` | `member_views.py`：`MemberUpdate`・`MemberDelete` | 権限あり |

### 班員（班のメンバーと、班の工数）（`frontend/src/TeamPage/`）

| 画面（タブの名前）と URL | 画面のファイル（`.tsx`） | 呼ぶ API（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| 班員Menu `/team-menu` | `TeamMenu` | `team_menu` | `main_views.py`：`TeamMenu` | 権限あり |
| 班員登録 `/team-new` | `TeamNew` | `team_new` | `team_views.py`：`TeamNew` | 権限あり |
| 班員工数履歴 `/team-list` | `TeamList` | `team_list` | `team_views.py`：`TeamList` | 権限あり |
| 班員工数履歴詳細 `/team-detail/:id` | `TeamDetail` | `team_detail` | `team_views.py`：`TeamDetail` | 権限あり |
| 班員工数入力状況 `/team-calendar` | `TeamCalendar` | `team_calendar`・`team_calendar_week_jump` | `team_views.py`：`TeamCalendar`・`TeamCalendarWeekJump` | 権限あり |
| 班員残業 `/team-overtime` | `TeamOverTime` | `team_overtime` | `team_views.py`：`TeamOverTime` | 権限あり |
| ショップ単位工数入力状況 `/team-view` | `TeamView` | `team_view`・`team_shop_select`・`team_export` | `team_views.py`：`TeamView`・`TeamShopSelect`・`TeamExport` | 権限あり |

### 問い合わせ（`frontend/src/InquirPage/`）

| 画面（タブの名前）とURL | 画面のファイル（`.tsx`） | 呼ぶ API（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| 問い合わせMenu `/inquir-menu` | `InquirMenu` | `inquir_menu` | `main_views.py`：`InquirMenu` | 全員 |
| 問い合わせ `/inquir-new` | `InquirNew` | `inquir_new` | `inquiry_views.py`：`InquirNew` | 全員 |
| 問い合わせ履歴 `/inquir-list` | `InquirList` | `inquir_list` | `inquiry_views.py`：`InquirList` | 全員 |
| 問い合わせ内容 `/inquir-detail/:id` | `InquirDetail` | `inquir_detail` | `inquiry_views.py`：`InquirDetail` | 全員 |
| 問い合わせ編集 `/inquir-update/:id` | `InquirUpdate` | `inquir_update` | `inquiry_views.py`：`InquirUpdate` | 全員 |

## 管理者（`frontend/src/AdministratorPage/`）

| 画面（タブの名前）とURL | 画面のファイル（`.tsx`） | 呼ぶAPI（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
|---|---|---|---|---|
| 管理者Menu `/manager-menu` | `AdministratorMenu` | `manager_menu` | `main_views.py`：`AdministratorMenu` | 管理者 |
| 設定編集 `/manager-update` | `AdministratorUpdate` | `manager_update` | `main_views.py`：`AdministratorUpdate` | 管理者 |
| データ管理 `/manager-loading` | `AdministratorLoading` | `kosu_backup`・`member_load`・`History_delet` など19個（データの種類ごとの `…_backup`・`…_load`・`…_delet`）・`check_backup_status`・`download_backup`・`manager_Loading` | `main_views.py`：`AdministratorLoading` `asynchronous_views.py`：`check_task_status`・`download_file`・`backup` | 管理者 |
| 全工数管理 `/manager-kosu` | `AdministratorKosuList` | `manager_kosu` | `main_views.py`：`AdministratorKosuList` | 管理者 |
| 全工数編集 `/manager-kosu-update/:id` | `AdministratorKosuUpdate` | `manager_kosu_update` | `main_views.py`：`AdministratorKosuUpdate` | 管理者 |
| 非同期タスクデータ管理 `/manager-task` | `AdministratorTaskList` | `manager_task` | `main_views.py`：`AdministratorTaskList` | 管理者 |
| 非同期タスク詳細 `/manager-task-detail/:id` | `AdministratorTaskDetail` | `manager_task_detail` | `main_views.py`：`AdministratorTaskDetail` | 管理者 |
| データ操作履歴一覧 `/manager-history` | `AdministratorHistoryList` | `manager_history` | `main_views.py`：`AdministratorHistoryList` | 管理者 |

| 画面（タブの名前）とURL | 画面のファイル（`.tsx`） | 呼ぶAPI（`/api/…/`） | サーバーの処理（`kosu/views/`） | 使える人 |
| --- | --- | --- | --- | --- |
| データ操作履歴詳細 `/manager-history-detail/:id` | `AdministratorHistoryDetail` | `manager_history_detail` | `main_views.py`：`AdministratorHistoryDetail` | 管理者 |

## 7.3 直したい物から、開くファイルを探す

| 直したい物 | 開くファイル | くわしくは |
|---|---|---|
| 画面の文字・ボタン・並び | §7.2 の「画面のファイル」（`frontend/src/…/…tsx`） | 05 章 |
| 画面の色・大きさ・余白 | `frontend/src/styles/` の、画面ごとのフォルダの CSS と `global.css` | 03 章 |
| どの画面でも使う部品（ページ送り・表・選ぶ欄・読み込み中の表示） | `frontend/src/Components/` | 05 章 |
| 画面の URL とタブの名前 | `frontend/src/index.tsx` | 05 章 |
| 一覧の並び順・しぼり込み・計算・権限の確かめ | §7.2 の「サーバーの処理」（`kosu/views/…py`） | 07 章 |
| 工数の 5 分刻みの書き込み・休憩の差し引き・入力が正しいかの判定 | `kosu/utils/kosu_utils.py` | 08 章 §8〜§14 |
| 一覧の 1 ページの件数（ページ送り） | `kosu/utils/main_utils.py` の `CustomPagination`（件数は 設定編集 の「一覧表示項目数」） | 07 章 |
| ショップ単位の Excel（「〇年〇月度_工数入力状況_〇.xlsx」） | `kosu/views/team_views.py` の `TeamExport` と `kosu/utils/team_utils.py` の `excel_function` | 07 章 |
| データ管理のバックアップ・ロード・削除の中身 | `kosu/tasks.py`（受付は `kosu/views/asynchronous_views.py`） | 09 章 §5〜§10 |
| 表の列（データの項目） | `kosu/models.py` を直して `makemigrations`（画面に出すなら `kosu/views/serializers.py` と画面のファイルも） | 08 章・07 章 |
| API の住所 | `kosu/urls.py`（いちばん外側は `hozen_another/urls.py`） | 07 章 |
| データの変更の履歴を残すしくみ | `kosu/signals.py` | 07 章 |
| 許可するホスト・データベース・ログなどの設定 | `hozen_another/settings.py` | 01 章 §11・09 章 §11 |
| 本番に出す手順 | `.github/workflows/master_hozen-kosu-react.yml` | 09 章 §14 |
| 画面が呼ぶサーバーの住所 | `frontend/.env.production`（本番）と `frontend/.env`（手元） | 01 章 §17 |
| テスト | `frontend/src/tests/` と `kosu/tests/test_backend.py` | 12 章 |

# 8. 気を付けること

## 8.1 手元でだけ書き換えて、コミットしない 2 か所

手元の PC でアプリを試す時に、次の所を書き換えることがあります。**書き換えたままコミットして `master` に push すると、（GitHub Actions が動いている時は）そのまま本番が変わります。**

| ファイル | `master`（本番）のまま | 手元で試す時だけ | 戻し忘れて push すると |
|---|---|---|---|
| `hozen_another/settings.py`（166〜167 行目） | `SESSION_COOKIE_SAMESITE = 'None'` と `SESSION_COOKIE_SECURE = True` | `'Lax'` と `False`（暗号化していない `http://localhost` でも、ログインが保たれるように） | 本番のクッキーが「暗号化していない通信でも送る」設定に変わる |

**コミットの前に、毎回これを確かめます:**

1. `git status` を打つ。
2. 上のファイルが「modified」と出ていたら、`git diff ファイル名` で中身を見る。
3. 手元でだけ書き換えた物なら、`git restore ファイル名` で元に戻す（02 章 §16）。直したい変更と混ざっている時は、その行だけを手で戻す。

> **`frontend/.env.production` は、今は読まれません。**画面のビルドが Vite に変わった（2026 年 9 月）ので、`frontend/.env.production`・`frontend/.env` の `REACT_APP_API_BASE_URL` は使われなくなりました（01 章 §17.2 は、前の Create React App の時の説明です）。手元で `npm start` で動かす方法は、15_直すべき所と直し方 の D-4 にあります。
>
> **書き換えなくて済むように直せます。**直し方は 15_直すべき所と直し方 の S-5（Cookie を `Lax` にする）と D-4（Vite の中継）にあります。

## 8.2 押す前に、必ずバックアップを取る操作

次の操作は、データが消えたり置き換わったりします。押す前に、**管理者 ⇒ データ管理** で、関係する種類のデータの「バックアップ開始」を押して、Excel を保存しておきます。

| 画面・ボタン | 何が起きるか | 戻すには |
|---|---|---|
| **ヘルプ（`/help`）の初期化** | パスワード（`HELP_PATH`）を入れた後、選んだ物（人員・工数区分定義・設定）を全部消し、初めの 1 件だけにする。人員を選ぶと、従業員番号 `12345`（名前 `admin`）の 1 人だけになり、ほかの人はだれもログインできなくなる（本番で今動いている 9 月の版では、画面の URL の誤りで、この初期化は動かない。`master` では 2026 年 11 月 9 日に直った。15 巻の B-1） | 事前のバックアップをロードする。無ければ Azure のデータベースの時点復元（09 章 §13.3） |
| **データ管理の「削除開始」** | 工数・タスク履歴・操作履歴を、選んだ期間の分だけ消す | 事前のバックアップをロードする |
| **データ管理の「ロード開始」** | 選んだ Excel をデータベースに入れる。入れ方は種類ごとに違う（下の表） | 事前のバックアップをロードする |
| **人員の削除** | その人が消える。その人の工数・問い合わせ・操作履歴の「氏名」のつながりが空になる（従業員番号は残る） | 人員を作り直しても、空になったつながりは戻らない |
| **工数区分定義の削除** | その版で入力した過去の工数を編集しようとすると「工数区分データが存在しません。」になる。全部の版を消すと、だれもログインできない（ERROR052） | 工数区分定義のバックアップをロードする |

**ロードの入れ方（種類ごと）：**

| 入れ方 | 種類 | 何が起きるか |
|---|---|---|
| 上書き | 工数・工数区分定義 | 同じ物（工数は「同じ従業員番号の同じ就業日」、定義は「同じ Ver 名」）があれば消してから入れる |
| スキップ | 人員・班員 | 同じ従業員番号がすでにあれば、その行は入れない（今のデータは変わらない） |
| 全部入れ替え | 作業詳細選択肢・設定 | 今のデータを全部消してから入れる |
| 足すだけ | 問い合わせ | 全行を新しく足す（同じファイルを 2 回ロードすると、2 件ずつになる） |

> くわしくは 09 章 §9。ロードした後は、管理者 ⇒ 非同期タスクデータ管理 で、結果（成功・失敗）を確かめます。

## 8.3 自分を締め出さない

- 自分の「管理者権限」を外したり、自分の人員データを消したりすると、管理者のメニューに入れなくなります。**管理者権限の人は、いつも 2 人以上** にしておきます。

- 工数区分定義は、少なくとも 1 つは残します（全部消すと、だれもログインできません）。同じ Ver 名の定義を 2 つ作るのも避けます（工数の編集で「複数の工数区分データが存在します。」になります）。

### 8.4 秘密の値

- `.env`（手元の秘密の値）は、コミットしません。ルートの `.gitignore` に入っているので、ふつうはコミットされません（02 章 §14）。
- もし秘密の値がもれたら、すぐに値を変えます（§3 の「秘密の値の受け取り方」のとおり、使っている所を全部）。Git の記録に一度入った値は、後から消しても「もれた物」として扱います。

### 8.5 新しい画面や API を足す時

- **サーバー（`kosu/views/`）でも、ログインと権限を確かめます。**画面でボタンを隠すだけでは、API の住所を直接呼ばれると防げません。今は `kosu/permissions.py` の共通の確かめが、全部の API に最初からかかっています（何も書かなければ「ログインが要る」）。「権限」が要る API には `permission_classes = [HasAuthority]`、管理者だけの API には `[IsAdministrator]` を書きます。さらに、他人のデータを扱う API では、その API の中で「そのデータが本人の物か」も確かめています（2026 年 11 月 10 日に全部そろった。15 巻の S-3）。新しく API を足す時も、同じように確かめてください。
- 画面を足したら、`frontend/src/index.tsx` に URL とタブの名前を、`kosu/urls.py` に API の住所を足します。
- テストも足します（12 章）。

> **直すべき所の一覧:** 今のアプリ（2026 年 11 月 11 日の `master`）で、安全・不具合・整理の面で直したい所は、場所（ファイルと行）と直し方つきで 15_直すべき所と直し方 に全部まとめてあります。今の `master` は、アプリのテストが 9 個落ちる状態です（15 巻の T-5）。引き継いだら、「🔴 すぐ」の物から直してください。

---

## 9. よくある依頼と、対応のしかた

### 9.1 新しく入った人を登録したい

1. 「権限」のある人が、人員 ⇒ 人員登録 を開く。
2. 従業員番号・氏名・ショップと、直ごとの休憩時間を入れて登録する（休憩時間の書き方は 08 章 §7）。
3. 班に入れる時は、その班の班長（「権限」のある人）が自分の番号でログインし、班員 ⇒ 班員登録 で、メンバーの欄にその人を選ぶ（1 つの班に 15 人まで）。選べるのは班長と同じショップの人で、班長のショップが「組長以上(P,R,T,その他)」「組長以上(W,A)」なら全員から選べます。

## 9.2 異動・退社した人

- **消さずに残す時:** 人員一覧 ⇒ 編集 で、ショップを「異動・退社」に変えます。人員一覧と、班員登録でメンバーを選ぶ欄で、いちばん後ろに回ります。過去の工数の「氏名」のつながりも残ります。ただし、人員の表に残っているので、その番号でログインはできます。
- **消す時:** 先に人員と工数のバックアップを取ってから、人員削除 で消します。その人の過去の工数・問い合わせ・履歴の「氏名」のつながりが空になります（§8.2）。
- どちらにするかは職場の決まりなので、前任者に確かめます（§10）。

## 9.3 「ログインできない」と言われた

| 出た文 | 原因 | すること |
|---|---|---|
| 「入力された従業員番号は登録がありません。ERROR048」 | 人員の表に、その番号が無い | 人員一覧で番号を探す。無ければ登録する（§9.1）。打ち間違いも確かめる |
| 「利用可能な工数区分がありません。ERROR052」 | 工数区分定義が 1 つも無い | 管理者が工数区分定義を登録するか、バックアップをロードする |
| ログインしても、すぐログインの画面に戻る | 本番なら、サーバーの不調やクッキーの設定の変化。手元なら §8.1 の 2 つ目 | 本番なら、他の人も同じか確かめ、09 章 §16（障害対応）へ |

## 9.4 工数区分（作業の種類）を変えたい（年度替わりなど）

1. 先に、工数区分定義のバックアップを取る。
2. 管理者が、工数区分定義 ⇒ 工数区分定義登録 で、**新しい Ver 名** の定義を作る（1 つの版に 50 区分まで）。古い版は消さずに残す（過去の工数がその版を使っているため。§8.2）。
3. ログインの時は、いちばん新しく作った（またはロードした）版が自動で選ばれます。使う版を変えたい人は、工数区分定義 ⇒ 工数区分定義切り替え で選びます。

## 9.5 作業詳細の選択肢を足したい

管理者が、工数区分定義 ⇒ 作業詳細選択肢登録 で足します。一覧・編集・削除は 作業詳細選択肢一覧 から行います。

## 9.6 一覧の 1 ページの件数を変えたい

管理者 ⇒ 設定編集 の「一覧表示項目数」を変えます（プログラムの変更は要りません）。

## 9.7 問い合わせに答える

1. 問い合わせ担当者のメインメニューに「〇〇さんからの新しい問い合わせがあります。」と出るので、押して内容を開く。
2. 問い合わせ内容 ⇒ 編集 で「回答」を書いて保存する（回答の欄は、管理者権限の人だけに出ます）。
3. 問い合わせた人のメインメニューに「ID〇の問い合わせに回答が来ています。」と出ます。

4. 不具合の報告なら、§7 の表で画面とファイルを探して直し、§6 の手順で本番に出します。

## 9.8 本番のエラーを調べたい

- 管理者 ⇒ 設定編集 の「バックエンドログ表示:」にチェックを入れると、画面の下に、本番のサーバーの記録（`web_console.log`）が 5 秒ごとに表示されます。記録は 1MB を超えると古い物から消えるので、障害の時は早めに中身を写しておきます（09 章 §11.10）。
- サーバーそのものが止まっている時（画面が開かない・502 が出る）は、Azure ポータルの App Service のログを見ます（09 章 §15・§16）。

## 9.9 データを Excel で出したい

- 種類ごとのデータ: 管理者 ⇒ データ管理 の「バックアップ開始」（工数・タスク履歴・操作履歴は期間を選ぶ）。
- ショップごとの、月の工数の入力状況: 班員 ⇒ ショップ単位工数入力状況 の「Excelに出力」（「〇年〇月度_工数入力状況_〇.xlsx」）。

## 9.10 画面の文字や見た目を直したい

1. §7.2 の表で、その画面の「画面のファイル」を探す。
2. 手元で直し、`npm start` で確かめる（05 章）。
3. §6 の手順で本番に出す。

## 9.11 表に列（データの項目）を足したい

1. `kosu/models.py` に列を足し、手元で `python manage.py makemigrations` と `python manage.py migrate`（08 章・07 章）。
2. 画面に出すなら、`kosu/views/serializers.py` と画面のファイルも直す。データ管理のバックアップ・ロードにも入れるなら `kosu/tasks.py` も直す。
3. §6 の手順で本番に出し、本番のデータベースに migrate する（**先にバックアップ**。やり方は前任者に確かめる。09 章 §14.3）。

## 9.12 古いデータを整理したい

1. 先に、消す期間のバックアップを取り、Excel を保管する。
2. 管理者 ⇒ データ管理 の「削除開始」で、期間を選んで消す。
3. なお、タスク履歴（`AsyncTask`）は 1,000 件、操作の履歴（`History`）は 50 万件を超えると、古い物から自動で消えます（`kosu/models.py` の `MAX_RECORDS`）。

---

## 10. 前任者に確かめること

次のことは、アプリのプログラムやこの資料からは分かりません。引き継ぎの時に前任者に聞いて、「答え」の欄を埋めておきます（この表は、手元に写して使ってください）。

| ☐ | 確かめること | なぜ要るか | 答え |
|---|---|---|---|
| ☐ | 本番のデータベースに **migrate する方法**（どこで・どのコマンドで） | GitHub Actions は migrate をしない（09 章 §14.3） | |
| ☐ | 本番の App Service の **起動のコマンド** と **環境変数** の場所 | リポジトリに書いていない（09 章 §14.1） | |
| ☐ | **ステージング**（`hozen-kosu-another`）は今も使うか。使うなら、どう出すか | GitHub Actions は本番にしか出さない（09 章 §2） | |
| ☐ | 本番で Django-Q のワーカー（`qcluster`）を動かしているか | `settings.py` に設定はあるが、本番の動かし方はリポジトリに無い（09 章 §3.4） | |
| ☐ | Django の管理画面（`/admin/`）のアカウントが本番にあるか | データを画面で直接見る時に要る | |
| ☐ | Azure のデータベースの自動バックアップを、何日分残しているか | 事故の時に、どこまで戻せるか（09 章 §13.3） | |
| ☐ | アプリ内バックアップ（データ管理）を、決まった日に取っているか。Excel をどこに保管しているか | 戻す時に要る | |
| ☐ | 年度替わりなど、毎年決まってやる作業（工数区分定義の更新など） | 忘れると現場が困る | |
| ☐ | 異動・退社した人を、消すか・「異動・退社」にするか（§9.2） | 職場の決まり | |
| ☐ | 止める時・不具合の時に、現場のだれに・どう知らせるか | 障害対応（09 章 §16） | |
| ☐ | Azure の費用・契約の担当と、障害の時の社内の連絡先 | お金と責任の窓口 | |
| ☐ | まだ答えていない問い合わせ、やりかけの改修 | 引き継いだ日から続きをやるため | |
| ☐ | これまでに起きた障害と、その時どうしたか | 同じことが起きた時のため | |

## 11. この巻のまとめ

- 本番は Azure、プログラムは GitHub の `master`。**`master` に push すると、GitHub Actions がそのまま本番に出します**（migrate はしない）。
- 使う人は、従業員番号だけでログインします。「権限」と「管理者権限」の 2 つのスイッチで、使えるメニューが増えます。
- 受け取る物（§3）と、前任者に確かめること（§10）を、引き継ぎの日に全部そろえます。
- 「この画面を直したい」は §7 の表から。直す前にバックアップ、押す前にバックアップ（§8.2）。
- 手元でだけ書き換えた 2 か所（§8.1）は、コミットの前に必ず戻します。

次は「00a_はじめての方へ_超入門」と「00_導入と全体像」に進み、§5 の進め方に沿って読んでいきましょう。